

Proyecto 1: Reconocimiento de Figuras

Reporte Técnico

Integrantes:

Isauro Trinidad Cynthia
Márquez Arias Ana Ixchel
Montiel Solorio Diego Gael
Rodríguez López Luis Fernando

Modelado y Programación
1 de octubre de 2026

1. Definición del problema

- **Entrada:** La ruta a una imagen en formato *.bmp* que contiene una o más figuras geométricas de colores sólidos, sin traslaparse, sin degradados y sobre un fondo uniforme.
- **Salida:** Reporte en consola para cada figura detectada en la imagen, determinando:
 - **Su color:** En formato hexadecimal (*#RRGGBB*).
 - **Su clasificación:**
 - **C:** Cuadriláteros.
 - **T:** Triángulos.
 - **O:** Círculos.
 - **X:** Otros (cualquier figura fuera de las categorías anteriores).

Ejemplo de salida en consola

```
[+] Procesando imagen: pruebas/figuras_01.bmp
-----
Figura 1 | Tipo: T (Triangulo)      | Color: #FF0000
Figura 2 | Tipo: C (Cuadrilatero)     | Color: #00FF00
Figura 3 | Tipo: O (Circulo)             | Color: #0000FF
Figura 4 | Tipo: X (Otro)              | Color: #FFFF00
-----
Procesamiento finalizado con exito.
```

2. Arsenal

2.1. Requisitos e instalación

El proyecto requiere Python descargado junto con las bibliotecas OpenCV y NumPy. Para instalarlas:

```
pip install opencv-python numpy
```

2.2. Lenguaje de programación

- **Python:** Nos decidimos por Python como lenguaje de desarrollo por su sintaxis sencilla, clara y versátil, la cual facilita el trabajo colaborativo, la comprensión del código entre los integrantes del equipo y la división del sistema en módulos independientes. Asimismo, su gestión automática de memoria evita la complejidad de manejo de punteros presente en otros lenguajes. Además, cuenta con un sólido ecosistema de bibliotecas especializadas en procesamiento y análisis de imágenes, lo que permite manipular píxeles e identificar las características necesarias para detectar y clasificar las figuras geométricas en archivos *.bmp*.

Bibliotecas:

- **OpenCV (cv2):** Se eligió porque, al ser una herramienta estándar en visión por computadora, es indispensable para procesar la imagen, filtrar el ruido y detectar las formas geométricas de cada figura.
- **NumPy (numpy):** Se eligió por su eficiencia para manipular las imágenes como matrices de píxeles, lo que permite realizar los cálculos matemáticos necesarios para separar el fondo y analizar los colores.
- **Bibliotecas estándar de Python (os, glob, sys):** Se ocuparon para gestionar la lectura de argumentos desde la terminal, validar las rutas de los archivos *.bmp* e inspeccionar el banco de pruebas.

3. Análisis del problema

3.1. Supuestos

Mismos supuestos dados en el detallamiento del proyecto.

- El fondo de la imagen es de un solo color, uniforme.
- Cada figura tiene un color sólido, distinto entre figuras y distinto del fondo.
- No hay anti-aliasing ni colores intermedios en los bordes de las figuras.
- Las figuras no se traslapan entre sí.
- Tamaño rotación y posición de las figuras son arbitrarios.

3.2. Casos límite identificados

- La imagen no cumple con las especificaciones del problema (sea una fotografía/imagen compleja con elementos distintos a las figuras esperadas). → Debe ser detectado por el programa y responder con un mensaje de error claro ('Imagen inválida') en vez de falle o dar una clasificación sin sentido.
- La imagen no está en el formato adecuado (no es *.bmp*). → De igual manera se debe detectar ya que no se puede admitir esa entrada y responder con un mensaje ('Formato inválido').
- Imagen sin ninguna figura, solo un fondo de color sólido.

3.3. Requisitos funcionales

- Lectura de imágenes mediante la dirección de un archivo *.bmp*
- Detección de figuras (C, T, O, X)
- Detección de colores (formato hexadecimal).
- Validación de entrada (formato de archivo e imagen)
- Algoritmo de búsqueda/clasificación de figuras
- Reporte de resultados al usuario: De forma clara y breve.

3.4. Requisitos NO funcionales

- Rendimiento: El programa debe procesar una imagen de tamaño razonable en un tiempo aceptable.
- Usabilidad: El programa regresa resultados comprensibles para el usuario.
- Mantenibilidad: El código debe estar organizado en secciones con tareas bien definidas.
- Robustez: El programa debe manejar entradas inválidas sin terminar de manera inesperada.
- Portabilidad: El programa debe poder ejecutarse en los entornos definidos del proyecto.
- Documentación del código: El código debe tener documentación suficiente para facilitar la comprensión y futuras modificaciones.

4. Selección de la mejor alternativa

Se eligió un algoritmo determinista de segmentación por distancia euclidiana BGR evaluada respecto a la moda del fondo:

$$d = \sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}$$

El flujo limpia la imagen mediante apertura morfológica ('cv2.MORPH_OPEN'), aproxima los vértices con el algoritmo de Ramer-Douglas-Peucker ('cv2.approxPolyDP') y calcula la métrica de circularidad isoperimétrica:

$$C = \frac{4\pi \cdot \text{Área}}{\text{Perímetro}^2}$$

Se prefirió este método sobre la binarización fija porque se adapta dinámicamente a cambios en el fondo sin umbrales estáticos. Asimismo, se descartó la Transformada de Hough por su elevado costo computacional y rigidez ante polígonos de orientaciones arbitrarias, y se rechazó la Inteligencia Artificial por exigir entrenamiento previo, recolección de datos y recursos de cómputo innecesarios para formas geométricas simples.

5. Diagrama de flujo o pseudocódigo

